

Document de Justification Technique

TP3 — Base de données, Doctrine ORM & CRUD complet

Projet : SchoolPrepar | UE : IT 232 — Développement Web II

Niveau : Licence | Filière : GL & WIM | Année : 2025-2026

1. Présentation du modèle de données

Le modèle de données de SchoolPrepar est conçu pour répondre aux besoins du cahier des charges : centraliser les informations sur les filières et établissements, et gérer des éléments dynamiques (événements). Trois entités principales ont été identifiées et implémentées avec Doctrine ORM.

Entités du modèle :

Entité	Table SQL	Rôle
Filiere	filiere	Représente une filière d'études (GL, WIM, RI...)
Etablissement	etablissement	Représente un établissement scolaire ou universitaire
Evenement	evenement	Élément dynamique : conférence, webinaire, JPO, salon...

2. Relations entre entités et justification

2.1 Relation ManyToMany — Filiere ↔ Etablissement (N:N)

Une filière peut être proposée par plusieurs établissements, et un établissement propose plusieurs filières. Cette relation N:N est matérialisée par une table pivot **filiere_etablissement** contenant les colonnes *filiere_id* et *etablissement_id*.

Justification : Dans la réalité, une même formation (ex : Génie Logiciel) est dispensée à IPNIT, ESGIS et UL simultanément. Une relation 1:N aurait été trop restrictive et ne refléterait pas la réalité du terrain.

Entité propriétaire	Entité inverse	Annotation Doctrine	Table pivot
Filiere	Etablissement	@ORM\ManyToMany + inversedBy	filiere_etablissement

2.2 Relation OneToMany — Filiere → Evenement (1:N)

Une filière peut être associée à plusieurs événements (webinaires, JPO...), mais un événement ne concerne qu'une seule filière à la fois. La clé étrangère **filiere_id** est stockée dans la table *evenement*.

2.3 Relation OneToMany — Etablissement → Evenement (1:N)

Un établissement peut organiser plusieurs événements, mais chaque événement n'a qu'un organisateur principal. La clé étrangère **etablissement_id** est stockée dans la table *evenement*.

Les deux colonnes *filiere_id* et *etablissement_id* dans la table *evenement* sont **nullable** (ON DELETE SET NULL), ce qui permet d'avoir des événements transversaux (ex : Salon de l'Étudiant sans filière spécifique).

3. Schéma relationnel (modèle physique)

```
filiere (id PK, nom, niveau, description, duree, debouches)
|
+-- filiere_etablissement (filiere_id FK, etablissement_id FK) [N:N pivot]
|
|   |-- etablissement (id PK, nom, sigle, ville, type, email, telephone)
|   |
|   |   |-- evenement.etablissement_id FK
|   |
|   +-- evenement (id PK, filiere_id FK, etablissement_id FK,
|                  titre, type, date_evenement, lieu, description)
```

4. CRUD complets implémentés

Trois CRUD complets ont été développés et intégrés dans le back-office /admin :

Entité	Controller	Routes	Opérations
Filiere	AdminFiliereController	/admin/filieres/*	Liste, Créer, Voir, Modifier, Supprimer
Etablissement	AdminEtablissementController	/admin/etablissements/*	Liste, Créer, Voir, Modifier, Supprimer
Evenement	AdminEvenementController	/admin/evenements/*	Liste, Créer, Voir, Modifier, Supprimer

Chaque CRUD respecte le pattern Symfony : Form Type dédié, protection CSRF sur les suppressions, flash messages de confirmation, et injection de l'EntityManager via le container.

5. Form Types Symfony

Trois Form Types ont été créés dans src/Form/ :

Classe	Entité liée	Particularité
FiliereType	Filiere	Champ EntityType multiple (checkboxes établissements)
EtablissementType	Etablissement	Champs texte simples, email, téléphone
EvenementType	Evenement	DateTimeType widget single_text + deux EntityType (filière et établissement)

6. Migration Doctrine

Le fichier de migration **Version20260413000001.php** dans le dossier migrations/ crée les 4 tables avec leurs contraintes de clés étrangères. Il est exécuté avec :

```
php bin/console make:migration
php bin/console doctrine:migrations:migrate
```

Pour les tests, les DataFixtures (src/DataFixtures/AppFixtures.php) chargent 6 établissements, 6 filières et 6 événements réalistes avec leurs relations :

```
composer require --dev doctrine/doctrine-fixtures-bundle
php bin/console doctrine:fixtures:load
```

7. Structure du projet après TP3

```
src/
  Controller/
```

AdminFiliereController.php	← CRUD Filiere
AdminEtablissementController.php	← CRUD Etablissement
AdminEvenementController.php	← CRUD Evenement (nouveau TP3)
FiliereController.php	← Front (utilise maintenant Doctrine)
EtablissementController.php	← Front (utilise maintenant Doctrine)
Entity/	
Filiere.php	← Entité Doctrine
Etablissement.php	← Entité Doctrine
Evenement.php	← Entité Doctrine
Form/	
FiliereType.php	
EtablissementType.php	
EvenementType.php	
Repository/	
FiliereRepository.php	
EtablissementRepository.php	
EvenementRepository.php	
DataFixtures/	
AppFixtures.php	← Données de test (6 enreg. / entité)
migrations/	
Version20260413000001.php	← Migration SQL complète

8. Choix techniques et justifications

Symfony 5.4 LTS : Version stable à long terme, compatible avec les bundles utilisés (Doctrine 2.x, Form).

Annotations Doctrine (@ORM...) : Approche moderne et lisible : les métadonnées sont co-localisées avec le code de l'entité, simplifiant la maintenance.

Nullable FK sur Evenement : Les relations filiere_id et etablissement_id sont optionnelles (ON DELETE SET NULL) pour permettre des événements transversaux.

CASCADE REMOVE sur collections : La suppression d'une Filiere ou d'un Etablissement supprime automatiquement ses événements liés, évitant les orphelins en base.

Protection CSRF sur suppressions : Chaque formulaire de suppression utilise csrf_token() pour prévenir les attaques CSRF, conformément aux bonnes pratiques Symfony.

Flash messages : Retour utilisateur après chaque opération CRUD via addFlash(), affiché dans le layout admin.

DataFixtures distinctes du code métier : Les données de test sont isolées dans src/DataFixtures/ et n'affectent pas l'environnement de production.